1 | Conventional middleware

This approach uses a RequestDelegate and an InvokeAsync method to process HTTP requests, optionally modify them, and pass control to the next component in the pipeline.

```
public class AnonymousUserMiddleware
{
    private const string AnonymousUserIdCookie = "AnonymousUserId";
    private readonly RequestDelegate _next;

    public AnonymousUserMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext context)
    {
        if (!context.Request.Cookies.TryGetValue(AnonymousUserIdCookie, out var userId))
        {
            userId = Guid.NewGuid().ToString();

            var cookieOptions = new CookieOptions
            {
                Expires = DateTime.UtcNow.AddMonths(1),
                IsEssential = true,
                HttpOnly = true
            };
            context.Response.Cookies.Append(AnonymousUserIdCookie, userId, cookieOptions);
        }

        // Store the user identifier in HttpContext for application-wide access
        context.Items[AnonymousUserIdCookie] = userId;

        // Continue with the pipeline
        await _next(context);
    }
}
```

Don't forget to register it:

```
// Without using extension method
app.UseMiddleware<AnonymousUserMiddleware>();
```

2 | Strongly Typed Middleware

IMiddleware interface provides the facility to introduce custom middleware.

```
public class AnonymousUserMiddleware : IMiddleware
{
    private const string CookieName = "AnonymousUserId";

    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        // Try to get existing anonymous user ID from cookie
        if (!context.Request.Cookies.TryGetValue(CookieName, out string? userId))
        {
            // Generate a unique identifier (GUID)
            userId = Guid.NewGuid().ToString();

            // Store the identifier in a cookie
            SetUserIdCookie(context, userId);
        }

        // Set the user identifier in the HttpContext for easy access in controllers
        context.Items[CookieName] = userId;

        // Continue with the request pipeline
        await next(context);
    }

    private static void SetUserIdCookie(HttpContext context, string userId)
    {
        var cookieOptions = new CookieOptions
        {
            Expires = DateTime.UtcNow.AddMonths(1),
            IsEssential = true,
            HttpOnly = true
        };

        context.Response.Cookies.Append(CookieName, userId, cookieOptions);
    }
}
```

Cleaner Configuration

Using an extension method to encapsulate middleware improves maintainability.

```
public static class AnonymousUserMiddlewareExtensions
{
    public static IApplicationBuilder UseAnonymousUserMiddleware(
        this IApplicationBuilder builder)
    {
        return builder.UseMiddleware<AnonymousUserMiddleware>();
    }
}
```

```
// NOTE: When using "IMiddleware" interface,
// the middleware should be registered as a service too.
builder.Services.AddTransient<AnonymousUserMiddleware>();

// Add the AnonymousUserMiddleware to the pipeline
app.UseAnonymousUserMiddleware();
```

3 | Inline Middleware

This approach uses a lambda with Use to execute custom logic before and after calling the next middleware in the pipeline.

```
app.Use(async (context, next) =>
{
    Console.WriteLine($"Request Path: {context.Request.Path}");

    // Call the next middleware in the pipeline
    await next.Invoke();

    Console.WriteLine(
        $"Response Status Code: {context.Response.StatusCode}");
});
```

To have a terminal middleware use Run which ends the pipeline and directly writes a response.

```
app.Run(async context =>
{
    await context.Response.WriteAsync(
        "This is the final middleware in the pipeline.");
});
```

Elliot One

Share and **Spread** Knowledge.